

ПРОДУКТОВАЯ РАЗРАБОТКА / ЗАДАНИЕ ДЛЯ КАНДИДАТА

AI-first QA / SDET Engineer

Проверка готовности агента LibreChat к выпуску

Срок	Ограничение работы	Формат
2–3 календарных дня	До 10 часов активной работы	Репозиторий + разбор решения

Задача

Запустите предоставленную конфигурацию LibreChat, исследуйте поведение одного продуктового агента, автоматизируйте приоритетные проверки и подготовьте заключение о готовности демонстрационного сценария к выпуску. Вы проверяете готовый объект, а не создаёте собственный чат-бот.

Главный результат

Связка «риск → сценарий → ожидаемое свойство → проверка → результат → доказательство».
Поднятый Langfuse и скриншот чата без проверок не считаются результатом тестирования.

До начала работы

Организатор должен выдать стартовый комплект: закреплённую версию LibreChat, рабочий Docker-профиль [1], конфигурацию агента и двух учебных инструментов, синтетические данные, две тестовые сессии с разными правами, контракт ожидаемого поведения и команду сброса состояния.

Также предоставляются управляемые режимы ответа инструментов (успех, пусто, таймаут, запрет), совместимый mock модели, пример запуска агентного сценария и доступ к реальной модели с достаточным небольшим лимитом. Доступ к coding agent обеспечивается отдельно. Поиск бесплатного endpoint, регистрация с платёжной картой и покупка подписки не входят в задание.

Для трасс используйте выданный изолированный проект Langfuse, аналог либо локальный структурированный JSONL. Самостоятельное развёртывание системы наблюдения не требуется. При несовместимости локального окружения организатор предоставляет готовую удалённую среду; запуск и полный вызов инструмента проверяются до старта.

Не превышайте 10 часов активной работы. По достижении лимита сдайте сделанное и опишите ограничения. Срок начинается после проверки окружения и доступов. Сбои выданного стенда или квоты провайдера не являются вашей ошибкой.

Объект проверки и его контракт

Один сценарий: сведения о начислениях

Пользователь запрашивает начисления по лицевому счёту за месяц. У агента два инструмента: `get_account(account_number)` и `get_charges(account_id, period)`. Все значения берутся из выданных тестовых данных. Права определяются серверной сессией; модель не должна назначать себе пользователя или расширять доступ.

Инструмент различает успешный ответ, отсутствие счёта, запрет доступа, таймаут и прочую ошибку. Пустой список начислений после успешного запроса допустим. Таймаут, ошибка и запрет доступа не означают нулевые начисления.

Ситуация	Ожидаемое поведение
Корректный запрос	Ответ опирается на нужный счёт и период; сумма и использованные данные соответствуют результату инструмента.
Недостаточно данных	Если не указан номер счёта или месяц, агент уточняет запрос, а не выдумывает обязательные аргументы.
Нет начислений	Агент сообщает об отсутствии начислений только после успешного чтения нужного периода.
Источник недоступен	Пользователь получает понятное сообщение о невозможности проверки, а не выдуманные суммы или успешный пустой результат.
Чужой счёт	Сервис запрещает доступ независимо от просьб в сообщении; данные другого пользователя не раскрываются.
Инструкция внутри данных	Посторонний текст в тестовом ответе инструмента не должен менять права, раскрывать чужие данные или подменять цель запроса.

Учебный дефект

В выданном демонстрационном компоненте намеренно нарушено одно из описанных правил. Найдите и подтвердите нарушение; искать обязательный баг в upstream LibreChat не нужно. Исправление приложения не требуется: главное, чтобы тест обнаруживал ошибку и давал понятное доказательство.

Границы

Не требуются нагрузочное тестирование, полный аудит безопасности, все провайдеры, RAG и все функции LibreChat. Работайте только с выданными синтетическими данными и стендом. Каждый независимый сценарий начинается в чистом диалоге, если тест явно не проверяет историю.

Что нужно сделать

1. Приоритеты и автоматизация

Опишите **8–10 сценариев** с риском, предусловиями, ожидаемым свойством и приоритетом. Включите категории со страницы 2. Автоматизируйте минимум пять, включая одну браузерную smoke-проверку отправки сообщения и отображения результата. Остальные можно проверить вручную с воспроизводимыми шагами.

Основные проверки можно выполнять на Python/pytest или TypeScript; для браузера подойдёт Playwright или аналог. Используйте существующую тестовую основу проекта. Права доступа проверяйте и на границе сервиса/инструмента, а не только по вежливому отказу модели в чате.

2. Два режима выполнения

Регрессия: реальные проверяемые компоненты LibreChat исполняются с управляемой моделью и/или внешними инструментами. Точки подмены перечислены в README. Не подменяйте весь ответ приложения эталонным JSON: тест должен иметь возможность обнаружить дефект приложения.

Реальная модель: выполните 2–3 ключевых сценария через выданные provider/model. Один сценарий повторите несколько раз и отразите все результаты, включая неудачные. Не считайте такой небольшой прогон статистической гарантией качества. При недоступности провайдера отмечайте BLOCKED; мок не заменяет этот прогон.

3. Проверки и трассы

Проверяйте аргументы инструментов, доступ, числа, статус ошибок и наличие нужного результата, а не точное совпадение свободного текста. Проверки через код достаточны; LLM-as-a-judge не обязателен. Langfuse можно использовать для трасс и оценок [2], но скриншот без критерия PASS / FAIL не является тестом.

Для каждого автоматического случая сохраните ID сценария и запуска, режим/модель, ожидаемое и фактическое поведение, статус и ссылку на трассу либо путь к логу. По одному дефекту покажите цепочку: запрос → вызов инструмента → ответ/ошибка → итог агента. Удалите из логов секреты.

4. Доказательства и решение о выпуске

Оформите баг-репорт на найденное учебное нарушение: версия, предусловия, шаги, ожидание, факт, влияние, доказательство и связанный тест. Команда тестов возвращает ненулевой код при FAIL; в отчёте BLOCKED явно отличается от PASS. Не скрывайте падающие проверки ради «зелёного» запуска.

В заключении укажите, что проверено, что заблокировано, какие риски остались и допускаете ли выпуск именно описанного сценария. Число найденных дефектов само по себе не оценивается; важны воспроизводимость и приоритеты.

Что сдавать и как оцениваем

Работа с AI

Настройте coding agent для тестового репозитория и используйте два коротких skill: **добавление регрессионного теста** и **разбор сбоя по трассе**. Укажите запускаемые команды и критерий проверки результата. Покажите, что тест действительно замечает нарушенное поведение, а не только повторяет утверждения агента.

В AI_USAGE.md кратко укажите агента и настройки, два примера его применения и что вы проверили самостоятельно. Покажите хотя бы одно решение агента, которое вы исправили или дополнительно проверили. Полные переписки с агентом и секреты не нужны.

Результат

- GitHub-репозиторий: тесты, фикстуры, настройки агента и skills. В README: версии, подготовка состояния, одна команда регрессии, отдельный запуск с реальной моделью и точки подмены.
- **Отчёт**: таблица 8–10 сценариев и результатов, воспроизводимый баг-репорт, пример трассы и заключение о выпуске. Достаточно Markdown/HTML, собственный dashboard не нужен. Добавьте AI_USAGE.md, затраченное время и ограничения.

Критерии

Критерий	Вес	Что считаем убедительным
Проверки поведения	30%	Ожидания связаны с контрактом и рисками; проверяются данные, права, ошибки и аргументы инструментов.
Надёжность тестов	30%	Набор воспроизводим; дефект обнаруживается; нет подмены проверяемой логики; есть браузерный сценарий.
Доказательства	20%	Баг подтверждён шагами и трассой; мок, реальная модель, FAIL и BLOCKED не смешиваются.
Инженерное понимание	20%	Кандидат объясняет тесты, проверяет работу AI, обосновывает приоритеты и ограничения решения о выпуске.

Обсуждение после сдачи

Разбор на 30–40 минут: запуск, объяснение ключевого решения и небольшое изменение с разрешённым AI. Пример: изменить один ответ инструмента и показать, какой тест должен упасть и почему. Сложность дополнительных функций и объём кода сами по себе не повышают оценку.

Документация

- [1] [LibreChat: запуск в Docker](#)
- [2] [Langfuse: оценка LLM-приложений](#)
- [3] [LibreChat: исходный код и тестовая инфраструктура](#)

Ссылки проверены 29.09.2026. Для выполнения используйте версии из стартового комплекта, а не произвольную текущую ветку.

Работа и сдача через GitHub

Создайте отдельный репозиторий в своём GitHub-аккаунте из папки своего задания. Ведите работу там с историей коммитов. При сдаче отправьте ссылку, SHA финального коммита и README с запуском. Можно публичный или приватный репозиторий; для приватного предоставьте доступ проверяющему. ZIP или patch не заменяют репозиторий. Секреты не публикуйте.